

指针与抽象

结构体、指针与STL容器 · 10题 · C++ & Python 双语对照

NQ091 替换空格 AcWing 16

请实现一个函数，把字符串中的每个空格替换成%20。

输入 输入一个字符串。数据范围： $0 \leq \text{输入字符串的长度} \leq 1000$

输出 输出替换后的字符串。

来源 AcWing 16

输入

We are happy.

输出

We%20are%20happy.

提示 注意输出格式。

C++

```
class Solution {
public:
    string replaceSpaces(string &str) {
        string res;
        for (auto c : str)
            if (c == ' ') res += "%20";
            else res += c;
        return res;
    }
};
```

Python

```
# AcWing 16
# Python solution
```

NQ092 AcWing 17 AcWing 17

AcWing 17

输入 见原题

输出 见原题

来源 AcWing 17

C++

```

/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:

    vector<int> printListReversingly(ListNode*
head) {
        vector<int> res;
        if (!head) return res;
        auto p = head;
        while(p) {
            res.push_back(p->val);
            p=p->next;
        }
        reverse(res.begin(), res.end());
        return res;
    }
};

```

Python

```

# AcWing 17
# Python solution

```

NQ093 用两个栈实现队列 AcWing 20

请用栈实现一个队列，支持如下四种操作： push(x) – 将元素 x 插到队尾； pop() – 将队首的元素弹出，并返回该元素； peek() – 返回队首元素； empty() – 返回队列是否为空。

输入

输入数据保证合法，例如，在队列为空时，不会进行 pop 或者 peek 等操作。数据范围：每组数据操作命令数量[0, 100]。

输出

输出每个操作的结果。

来源

AcWing 20

输入

```

MyQueue queue = new MyQueue();
queue.push(1);
queue.push(2);
queue.peek();
queue.pop();
queue.empty();

```

输出

```

1
1
false

```

提示 注意： 你只能使用栈的标准操作： push to top, peek/pop from top,size和is empty；如果你选择的编程语言没有栈的标准库，你可以使用list或者deque等模拟栈的操作；输入数据保证合法，例如，在队列为空时，不会进行pop或者peek等操作；

C++

```

class MyQueue {
public:
    stack<int> stk, cache;
    /** Initialize your data structure here. */
    MyQueue() {

    }

    /** Push element x to the back of queue. */
    void push(int x) {
        stk.push(x);
    }

    void copy(stack<int>& a, stack<int>& b){
        while (a.size()) {
            b.push(a.top());
            a.pop();
        }
    }

    /** Removes the element from in front of queue and returns that element. */
    int pop() {
        copy(stk,cache); //拷贝到cache数组
        int res = cache.top();
        cache.pop();
        copy(cache,stk);
        return res;
    }

    /** Get the front element. */
    int peek() {
        copy(stk,cache); //拷贝到cache数组
        int res = cache.top();
        copy(cache,stk);
        return res;
    }

    /** Returns whether the queue is empty. */
    bool empty() {
        return stk.empty();
    }
};

/**
 * Your MyQueue object will be instantiated and called as such:
 * MyQueue obj = MyQueue();
 * obj.push(x);
 * int param_2 = obj.pop();
 * int param_3 = obj.peek();
 * bool param_4 = obj.empty();
 */

```

Python

```

# AcWing 20
# Python solution

```

NQ094 斐波那契数列 AcWing 21

输入一个整数 n ，要求斐波那契数列的第 n 项。假定从0开始，第0项为0。

输入 输入一个整数 n 。数据范围： $0 \leq n \leq 39$

输出 返回斐波那契数列的第n项。

来源 AcWing 21

输入

5

输出

5

提示 注意输出格式。

C++

```
class Solution {
public:
    int Fibonacci(int n) {
        if (n <= 1) return n;
        return Fibonacci(n - 1) + Fibonacci(n - 2);
    }
};
```

Python

```
# AcWing 21
# Python solution
```

NQ095 反转链表 AcWing 35

定义一个函数，输入一个链表的头结点，反转该链表并输出反转后链表的头结点。

输入 输入一个链表的头结点。数据范围：链表长度[0, 30]。

输出 输出反转后的链表的头结点。

来源 AcWing 35

输入

1->2->3->4->5->NULL

输出

5->4->3->2->1->NULL

C++

```

/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        if (!head || !head->next) return head;

        auto p = head, q = p->next;
        while (q)
        {
            auto o = q->next;
            q->next = p;
            p = q, q = o;
        }
        head->next = NULL;

        return p;
    }
};

```

Python

```

# AcWing 35
# Python solution

```

NQ096 合并两个排序的链表 AcWing 36

输入两个递增排序的链表，合并这两个链表并使新链表中的结点仍然是按照递增排序的。

输入 输入两个链表。数据范围：链表长度[0, 500]

输出 输出合并后的链表。

来源 AcWing 36

输入

1->3->5
2->4->5

输出

1->2->3->4->5->5

提示 注意输出格式。

C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode* merge(ListNode* l1, ListNode* l2)
    {
        auto dummy = new ListNode(-1), tail = dummy;
        while (l1 && l2)
            if (l1->val < l2->val)
            {
                tail->next = l1;
                l1 = l1->next;
            }
            else
            {
                tail->next = l2;
                l2 = l2->next;
            }

            if (l1) tail->next = l1;
            if (l2) tail->next = l2;

            return dummy->next;
    }
};
```

Python

```
# AcWing 36
# Python solution
```

NQ097 AcWing 862 AcWing 862

AcWing 862

输入 见原题**输出** 见原题**来源** AcWing 862

C++

```
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 10010;

struct data {
    int x;
    double y;
    string z;
    bool operator< (const data & t) const
    {
        return x < t.x;
    }
} a[N];

int main() {
    int n;
    cin >> n;
    for (int i = 0; i < n; i++) cin >> a[i].x >> a[i].y >> a[i].z;
    // 结构体排序
    sort(a, a + n);
    // 输出结果
    for (int i = 0; i < n; i++)
        printf("%d %.2lf %s\n", a[i].x, a[i].y, a[i].z.c_str());
    return 0;
}
```

Python

```
# AcWing 862
# Python solution
```

NQ098 AcWing 810

AcWing 810

AcWing 810

输入

见原题

输出

见原题

来源

AcWing 810

C++

```
#include <iostream>

using namespace std;

int abs(int x)
{
    if (x > 0) return x;
    return -x;
}

int main()
{
    int x;
    cin >> x;
    cout << abs(x) << endl;

    return 0;
}
```

Python

```
# AcWing 810
# Python solution
```

NQ099 复制数组 AcWing 814

要编写函数 `void copy(int a[], int b[], int size)` 将 `a` 数组中的前 `size` 个数复制到 `b` 数组中并输出 `b` 数组。

输入 第一行 `n, m, size` 分别表示 `a` 数组长度、`b` 数组长度和要复制的个数；第二行 `n` 个整数为数组 `a`；第三行 `m` 个整数为数组 `b`。数据范围： $1 \leq n \leq m \leq 100$, $1 \leq size \leq n$

输出 一行 `m` 个整数表示复制后的数组 `b`。

来源 AcWing 814

输入

```
3 5 2
1 2 3
4 5 6 7 8
```

输出

```
1 2 6 7 8
```

提示 注意输出格式。来源：语法题

C++

```
#include <iostream>
#include <cstring>

using namespace std;

const int N = 110;

void copy(int a[], int b[], int size)
{
    memcpy(b, a, size * 4);
}

int main()
{
    int a[N], b[N];
    int n, m, size;
    cin >> n >> m >> size;
    for (int i = 0; i < n; i++) cin >> a[i];
    for (int i = 0; i < m; i++) cin >> b[i];

    copy(a, b, size);

    for (int i = 0; i < m; i++) cout << b[i]
<< ' ';
    cout << endl;

    return 0;
}
```

Python

```
# AcWing 814
# Python solution
```

NQ100 数组翻转 AcWing 816

编写函数 `void reverse(int a[], int size)` 实现将数组 `a` 中的前 `size` 个数翻转并输出翻转后的数组 `a`。

输入 第一行 `n` 和 `size` 分别表示数组长度和要翻转的个数；第二行 `n` 个整数为数组 `a`。数据范围： $1 \leq \text{size} \leq n \leq 1000$, $1 \leq a[i] \leq 1000$

输出 一行 `n` 个整数表示翻转后的数组 `a`。

来源 AcWing 816

输入

```
5 3
1 2 3 4 5
```

输出

```
3 2 1 4 5
```

提示 注意输出格式。来源：语法题

C++

```
#include <iostream>

using namespace std;

void reverse(int a[], int size)
{
    for (int i = 0, j = size - 1; i < j; i ++, j -- )
        swap(a[i], a[j]);
}

int main()
{
    int a[1000];
    int n, size;

    cin >> n >> size;
    for (int i = 0; i < n; i ++ ) cin >> a[i];
    reverse(a, size);

    for (int i = 0; i < n; i ++ ) cout << a[i]
<< ' ';

    return 0;
}
```

Python

```
# AcWing 816
# Python solution
```